

Python Collections

Lab Tasks: Lists, Tuples, Dictionaries and Sets

Instructions: Write and execute a Python program for each task below. Save each program with an appropriate filename, and verify the output before moving to the next task. Add comments to your code explaining key steps.

Part A Lists

A1. Creating Lists

- 1 Create a list of 10 integers of your choice and print the list along with its length.
- 2 Create a list containing at least five different data types (int, float, string, boolean, list) and print the type of each element.
- 3 Write a program to create an empty list, then add 5 elements to it one at a time using **append()** and print the final list.

A2. Accessing List Elements

- 4 Given a list of 8 fruit names, print the first element, the last element, and the element at index 3.
- 5 Write a program that accepts a list of numbers and prints every element along with its index using **enumerate()**.

A3. Slicing Lists

- 6 Given a list of 10 numbers, print the first 3 elements, the last 3 elements, and every alternate element using slicing.
- 7 Write a program to reverse a list using slicing (without using the **reverse()** method).
- 8 Given a list of 12 elements, extract and print the middle 4 elements using slicing.

A4. Negative Indices

- 9 Given a list of 7 elements, print the last element, the second-last element, and the last 3 elements using negative indexing.
- 10 Write a program that uses a negative step slice to print a list in reverse order.

A5. List Methods

- 11 Create a list of numbers and demonstrate the use of **append()**, **insert()**, **extend()**, **remove()**, **pop()**, **sort()**, **reverse()**, **count()**, and **index()** methods, printing the list after each operation.
- 12 Write a program to remove duplicate elements from a list without using the **set()** function.
- 13 Write a program to find the maximum, minimum, and sum of elements in a list without using built-in **max()**, **min()**, or **sum()** functions.

14 Write a program that merges two lists and sorts the combined list in descending order.

A6. List Comprehensions

15 Use a list comprehension to generate a list of squares of numbers from 1 to 20.

16 Use a list comprehension to create a list of all even numbers between 1 and 50.

17 Given a list of words, use a list comprehension to create a new list containing only the words with more than 4 letters.

18 Use a nested list comprehension to create a 3x3 matrix (list of lists) containing numbers 1 to 9.

19 Use a list comprehension with a conditional expression to replace every negative number in a list with 0, keeping positive numbers unchanged.

Part B Tuples

B1. Creating Tuples

- 1 Create a tuple containing the names of 6 countries and print it along with its type and length.
- 2 Write a program to create a tuple with a single element and verify its type (note the required comma).
- 3 Write a program that converts a list into a tuple and a tuple into a list.

B2. Indexing and Slicing Tuples

- 4 Given a tuple of 10 elements, print the element at index 0, index 5, and the last element using positive and negative indexing.
- 5 Given a tuple of 12 elements, use slicing to print the first half and the second half separately.
- 6 Write a program to check whether a given value exists in a tuple, using the **in** keyword.
- 7 Write a program to find the maximum and minimum values in a tuple of numbers, and count how many times a given value occurs using the **count()** method.

B3. Operations on Tuples

- 8 Write a program to concatenate two tuples and repeat a tuple 3 times using the **+** and ***** operators.
- 9 Write a program to unpack a tuple of 5 student marks into 5 separate variables and print their average.
- 10 Write a program that demonstrates tuple immutability by attempting to modify an element and catching the resulting error using try/except.
- 11 Given a tuple containing a nested list, modify the nested list and explain (in a comment) why this is possible despite tuples being immutable.
- 12 Write a program to sort the elements of a tuple in ascending order (result should be returned as a list, since tuples cannot be sorted in place).

Part C Dictionaries

C1. Creating, Adding, and Replacing Values

- 1 Create a dictionary to store the roll numbers and names of 5 students, then print the dictionary.
- 2 Write a program to add 3 new key-value pairs to an existing dictionary.
- 3 Write a program to update (replace) the value of an existing key in a dictionary and print the dictionary before and after the update.
- 4 Write a program that creates a dictionary from two separate lists (one of keys and one of values) using **zip()**.
- 5 Write a program to create a nested dictionary representing 3 employees, each with keys for 'name', 'department', and 'salary'.

C2. Operations on Dictionaries

- 6 Write a program to iterate through a dictionary and print all keys, all values, and all key-value pairs using **keys()**, **values()**, and **items()**.
- 7 Write a program to remove a key from a dictionary using **pop()**, and safely remove a key that may not exist using **get()** with a default value.
- 8 Write a program to check whether a given key exists in a dictionary before accessing it.
- 9 Write a program to merge two dictionaries into one using the **update()** method and the **|** operator (Python 3.9+).
- 10 Given a dictionary of item names and prices, write a program to find the item with the highest price and the item with the lowest price.
- 11 Write a program that counts the frequency of each character in a string and stores the result in a dictionary.
- 12 Use a dictionary comprehension to create a dictionary that maps numbers 1 to 10 to their cubes.

Part D Sets

D1. Creating Sets

- 1 Create a set of 8 elements (with at least 2 intentional duplicates) and print it to show that duplicates are automatically removed.
- 2 Write a program to create a set from a list and from a string, and print the results.
- 3 Write a program to add a single element to a set using **add()** and multiple elements using **update()**.

D2. Operations on Sets

- 4 Given two sets of numbers, write a program to find their **union**, **intersection**, **difference**, and **symmetric difference**.
- 5 Write a program to check whether one set is a subset or superset of another set using **issubset()** and **issuperset()**.
- 6 Write a program to remove an element from a set using **remove()** and **discard()**, and explain (in a comment) the difference in behavior when the element does not exist.
- 7 Write a program to check whether two sets are disjoint (have no elements in common) using **isdisjoint()**.
- 8 Given a list of numbers with duplicates, write a program to use a set to print only the unique elements, then convert the result back into a sorted list.
- 9 Use a set comprehension to create a set of squares of all odd numbers from 1 to 20.

End of Lab Tasks — Submit your source code files along with a screenshot of the output for each task.